

ECoMEM: Explicit Concept Memory for Memory-Dependent Robot Control

Yize Liu Ke Wang Mac Schwager Yiqing Xu[†] Jiajun Wu[†]

Stanford University

{yizeliu, yiqingx}@stanford.edu

[†]Equal advising

<https://ecomem.github.io/>

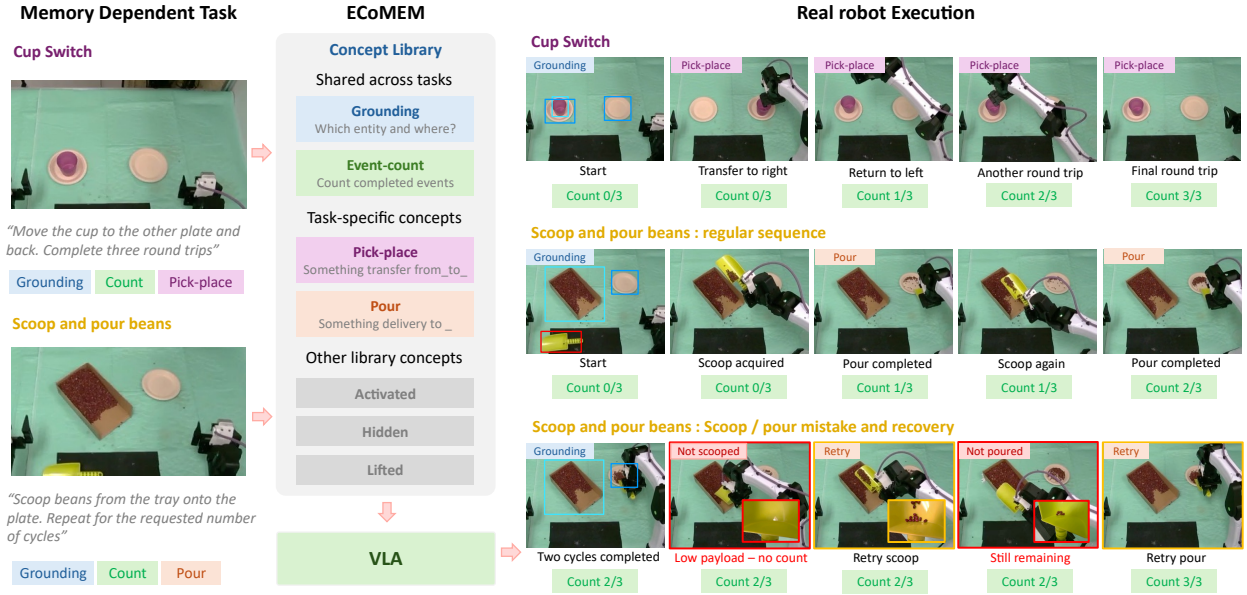


Figure 1: Explicit Concept Memory (ECoMEM) represents task-relevant history with compositional concept memory. **Left:** memory-dependent tasks require information that may no longer be available in the current observation. **Middle:** shared concepts for grounding and progress compose with task-specific event concepts. **Right:** the resulting memory conditions the VLA to track progress and recover from unsuccessful events on a real robot.

Abstract

A robot may lose sight of an object it must later retrieve, need to recall what a person demonstrated earlier, or track which steps of a task it has already completed. Current vision–language–action (VLA) policies often fail once the information needed for action disappears from the current observation, making memory critical for long-horizon robot behavior. Existing approaches typically provide longer histories or learn implicit memory from observation–action trajectories. But action supervision tells a policy *how to act*, not *what to remember*: it does not specify which past facts should persist or how they should change

as new evidence arrives. We therefore separate *maintaining an evidence-grounded account of the past* from *learning how to act on it*. This insight motivates **Explicit Concept Memory (ECoMEM)**, which represents task-relevant history with a reusable library of grounded concepts. An evidence-based Writer selects and updates these records, while a learned Reader turns them into memory tokens that directly condition the VLA. Across 16 RoboMME tasks, ECoMEM leads the evaluated robot policies on 15 tasks. On two new real-robot tasks, the same memory library either transfers directly or requires only one new concept, achieving 86.1% success versus 8.6% for a no-memory VLA. These results show that explicit concepts provide a reusable and extensible memory interface for robot control. [Project website](#).

1 Introduction

A robot that acts over extended periods cannot treat every observation as a fresh start. It may need to remember where an object was last seen after it becomes occluded, which object a person interacted with earlier, what parts of a task have already been completed, or the order in which previous events occurred. In each case, the information needed for the next action is no longer available in the current observation. Yet the relevant past is often much smaller than the full interaction history: what matters is a compact set of task-relevant facts about entities, states, events, and progress.

Vision–language–action (VLA) policies increasingly incorporate history through extended context, recurrent representations, learned memory banks, and test-time adaptation (Torne et al., 2025; Koo et al., 2026; Shi et al., 2026; Jiang et al., 2026). These mechanisms can preserve rich perceptual history, but they leave an important question implicit: *what from that history should become memory?* Action supervision specifies what the robot should do, but not which past facts should be retained, when a belief should be revised, or when two observations correspond to the same event. When memory is learned only through the control objective, the policy may instead rely on shortcuts in current observations, previous actions, or trajectory-specific correlations (de Haan et al., 2019; Wen et al., 2020; Zhao et al., 2025). Successful action prediction therefore does not by itself establish that the task-relevant history has been represented reliably.

Making memory explicit helps, but does not remove the problem of memory construction. Keyframe memories preserve visual evidence (Yang et al., 2026; Fang et al., 2025), leaving the policy to recover the relevant facts from stored images. VLM-based approaches summarize histories (Sridhar et al., 2026; Anwar et al., 2025), but place entity tracking, state estimation, and temporal interpretation inside long-video reasoning, where spatial and temporal errors remain challenging (Qi et al., 2025; Seth et al., 2025). Textual world states (Yoneda et al., 2024; Liu et al., 2023) and scene graphs (Gu et al., 2024; Rana et al., 2023; Liu et al., 2025) provide more structured representations, but a robotic memory must additionally decide how observations become persistent facts: what should survive occlusion, what should be revised, what constitutes a completed event, and how repeated evidence should be reconciled.

Our key idea is to separate *maintaining an evidence-grounded account of the past* from *learning how to act on it*. Rather than asking the policy to infer memory implicitly from history, we represent task-relevant history as explicit facts about entities, states, completed events, progress, and temporal order. These facts are updated only when supported by visual and robot-state evidence, while the policy learns how they should influence action. This separation also keeps task requirements distinct from observations: the instruction specifies what should be achieved, while memory records what has actually happened.

We instantiate this principle in **Explicit Concept Memory (ECoMEM)**, an explicit memory channel for VLA policies (Figure 2). First, a shared concept library provides reusable memory primitives spanning entity and spatial grounding, state and relations, events and progress, and temporal and procedural structure.

Second, an evidence-based *Writer* selects the concepts required by the instruction, grounds them to task entities, and maintains their records over time, while a learned *Reader* maps the resulting structured memory into latent tokens. Third, these memory tokens directly condition the VLA alongside visual and language inputs. Recognition and memory writing remain fixed during policy training, so evidence determines *what is remembered*, while action supervision learns *how that memory should be used*.

Across the 16 memory-dependent tasks of RoboMME (Dai et al., 2026), ECoMEM achieves the highest average success among the evaluated robot policies and leads on 15 of 16 tasks. Controlled experiments show that spatial grounding, event and progress information, temporal structure, task-conditioned selection, and language grounding each contribute to effective memory construction. More importantly, the same abstraction transfers to new physical-robot tasks: CUPSWITCH reuses the existing library without adding a new recognizer, while SCOOPPOUR requires only one new event concept composed with the existing memory machinery. Across shared real-robot conditions, ECoMEM succeeds in 68/79 trials (86.1%), compared with 5/58 (8.6%) for the no-memory $\pi_{0.5}$ baseline. These results suggest that explicit, compositional concepts can serve not merely as task-specific memory annotations, but as a reusable and extensible interface between a robot’s accumulated experience and learned control.

2 Related Work

2.1 Implicit memory in robot policies

History-aware robot policies typically learn a latent representation of past observations and actions. Long-context policies directly attend to extended trajectories (Torne et al., 2025; Koo et al., 2026); recurrent architectures propagate learned state across time (Li et al., 2026); memory-bank methods compress and retrieve past perceptual features (Shi et al., 2026); and test-time adaptation stores history in model weights (Jiang et al., 2026). Predictive world models similarly maintain latent state for control (Wu et al., 2023; Samsami et al., 2024). These approaches can capture rich temporal dependencies without prescribing what the representation must contain. However, what is retained, forgotten, or updated is learned largely through downstream objectives and remains difficult to inspect independently of the policy. ECoMEM instead externalizes a sparse set of task-relevant facts and fixes their evidence-based updates, while learning only how the policy should use them. Explicit and latent memory are therefore complementary rather than mutually exclusive.

2.2 Explicit memory in robot policies

A second line of work stores selected parts of experience explicitly. Keyframe and retrieval methods retain event-triggered images or trajectory segments (Yang et al., 2026; Fang et al., 2025; Memmel et al., 2025; Jain et al., 2024); MemER retrieves task-relevant visual experiences and uses a high-level VLM to condition a low-level policy (Sridhar et al., 2026). Other approaches summarize experience into language (Anwar et al., 2025; Liu et al., 2023), maintain textual world states (Yoneda et al., 2024), or organize persistent information in scene graphs (Gu et al., 2024; Rana et al., 2023; Liu et al., 2025). These representations improve access to past information, but visual memories still require the controller to recover facts from stored observations, while language and graph memories are commonly consumed by high-level reasoning or planning modules. ECoMEM instead maintains typed, evidence-linked records of states, events, progress, and temporal structure, and feeds them directly to a low-level VLA through a learned Reader.

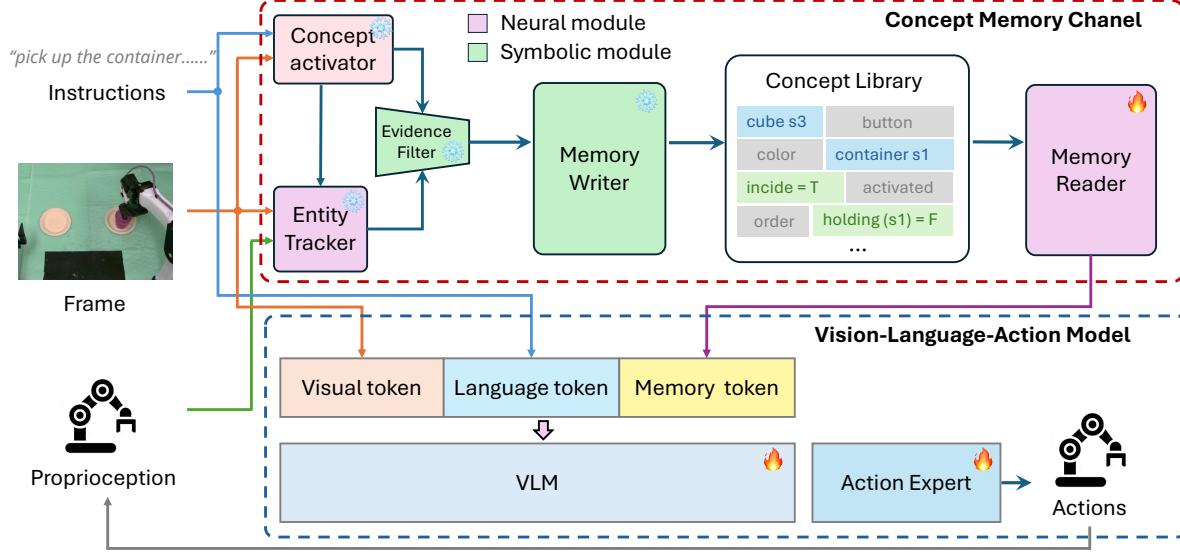


Figure 2: ECoMEM separates memory construction from memory use. The Writer selects task-relevant concepts, recognizes them from visual and robot-state evidence, and maintains structured records. The learned Reader encodes these records as memory tokens that condition the VLA alongside image and language tokens.

2.3 Neuro-symbolic methods in robotics

Neuro-symbolic robotics combines learned perception and control with explicit, compositional abstractions. Knowledge representations such as KnowRob organize persistent robot knowledge (Tenorth and Beetz, 2009); neuro-symbolic concept learning and structured world models ground symbolic concepts in perception and compose them for reasoning (Mao et al., 2019, 2022); and learned or invented predicates provide compact abstractions for task planning (Liang et al., 2025; Athalye et al., 2026; Silver et al., 2023). Related work also uses symbolic concepts to structure learned generative models for manipulation and geometric reasoning (Xu et al., 2024, 2025). Recent systems further use vision-language models to construct symbolic planning abstractions from demonstrations (Lorang et al., 2026), while knowledge graphs provide structured memory for language-model agents (Anokhin et al., 2025). ECoMEM follows the same principle of grounding reusable symbolic concepts in learned representations, but uses them for a different role: as the interface between perceptual history and a learned control policy.

3 Preliminaries

Memory-dependent tasks. Given an instruction ℓ , a VLA policy observes o_t (camera images and robot state) at decision t and predicts an action chunk a_t . The history h_t contains all observations and executed actions up to t , plus any demonstration given with the task. A task is *memory-dependent* if two histories with the same instruction and current observation require different expert actions:

$$\pi^*(\cdot \mid \ell, h_t) \neq \pi^*(\cdot \mid \ell, h'_t) \quad \text{with } o_t = o'_t. \quad (1)$$

Pressing a button three times illustrates this: the scene may look identical after the first and third press, yet the correct action differs. A policy conditioned only on (ℓ, o_t) cannot distinguish them.

Memory-conditioned policy. The history is long, but what separates h_t from h'_t is usually a few facts. We therefore compress the history into a memory m_t : a set of concept records, each stating one fact about the

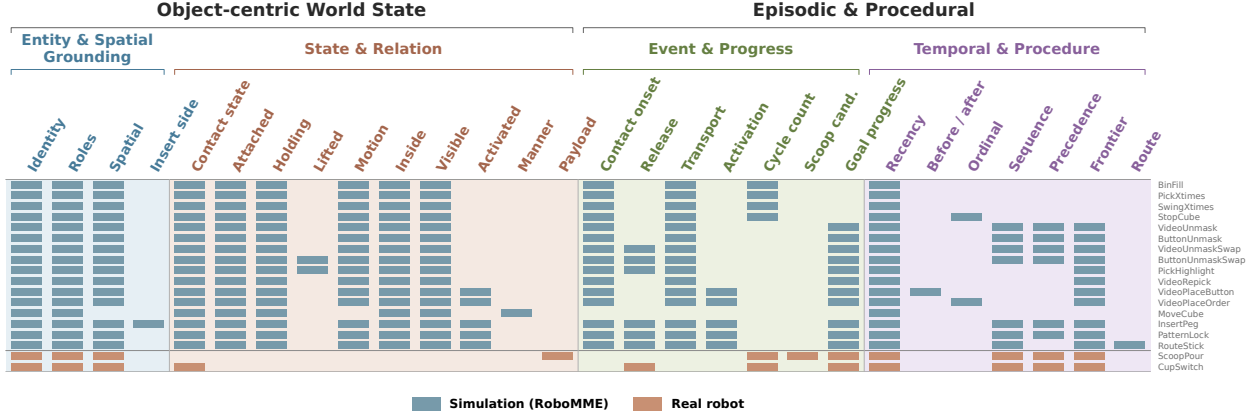


Figure 3: A shared concept library supports diverse memory tasks. Columns show concepts grouped into four semantic families, and rows show the 16 RoboMME tasks and two real-robot tasks. Filled cells indicate the concepts used by each task, revealing extensive reuse of shared concepts across otherwise different memory demands.

task entities (Figure 3). We separate what to remember from how to use it:

$$m_t = W(\ell, h_t), \quad u_t = R_\phi(m_t), \quad a_t \sim \pi_\theta(\cdot \mid \ell, o_t, u_t). \quad (2)$$

We construct a Writer W that selects concepts, harnesses pretrained perception models to recognize them, and updates records using only the history available at t (Section 4.2). We keep W fixed, because action supervision alone does not specify which facts to keep; a learned memory can instead exploit shortcuts (de Haan et al., 2019; Wen et al., 2020). We then train a Reader R_ϕ jointly with the VLA π_θ on expert action chunks a_t^* . The Reader encodes the records into memory tokens u_t that condition the policy (Section 4.3, Appendix A).

4 ECoMEM

ECoMEM adds an explicit memory channel to a VLA (Figure 2), following the factorization in Equation 2. The *Writer* W determines what to remember: it draws from a shared concept library (Section 4.1), selects the concepts required by the task, recognizes them from visual and robot-state evidence, and updates the memory records (Section 4.2). The *Reader* R_ϕ learns how to use this memory by encoding the records into tokens that condition the VLA (Section 4.3).

4.1 Concept Library

A shared concept library. Memory-dependent tasks differ, but the facts they need recur. Counting button presses and counting placements both need the number of completed events. Finding a hidden cube and re-picking a demonstrated object both need a remembered location. We therefore define each type of fact once, as a concept in a shared library. A new task reuses these concepts instead of building from scratch, and a single Reader serves all tasks. Robot knowledge bases and neuro-symbolic concept learners follow the same principle (Tenorth and Beetz, 2009; Mao et al., 2019).

Four concept families. We keep a concept only if it can change the correct action when the instruction and current observation are the same (Equation 1). The concepts that meet this criterion fall into four families (Figure 3). Each family resolves an ambiguity that the previous ones cannot. **Entity & Spatial Grounding** answers *what* and *where*. A remembered fact is useful only if it stays attached to the right object, so this

family binds facts to persistent entities and their locations (Gu et al., 2024). **State & Relation** answers *in what state*. Knowing the target does not tell the robot what to do with it, so this family records states and relations such as `holding` and `inside` (Silver et al., 2023). **Event & Progress** answers *what has happened*. The same scene can call for continuing or stopping, so this family records completed events and progress toward the goal (Huang et al., 2023). **Temporal & Procedure** answers *in what order*. Counts ignore order, so this family records the order of events and the required order of steps. The first two families describe the current world; the last two describe the past.

Concepts and records. We construct each concept with a coding agent. The library is built once and then reused in most of the other tasks. It specifies its arguments, a recognizer that harnesses pretrained perception models and robot state, and an update rule written as explicit logic. The library stores concepts; memory stores their instances as records. For example, the record `INSIDE(s2, s1)` states that entity `s2` is inside container `s1`, with a truth value, a status, a time, and its supporting evidence. Concepts also compose. The pick-place concept builds on the transport concept, which in turn builds on contact and region visits.

Extending the library. We build the library to grow with new tasks. Because concepts compose, a new concept reuses the existing perception models, lower-level concepts, and record format. An agent constructs the new concept on top of the library, and we verify it on a few demonstrations (Appendix I.2). For example, scooping beans into a bowl twice needs a scooping concept, which the library lacks. We could add `SCOOP` to the library: it reuses entity grounding, containment, and event counting, and adds only a check that the scooper carries beans (Section 5.5).

4.2 Constructing Concept Memory

The Writer W builds the memory m_t in three steps: it selects the concepts a task needs, recognizes them from the history h_t , and updates the records.

Task-conditioned selection and grounding. Running every concept is costly, and it adds records the task does not need, which distract the policy (Section 5.4). The Writer therefore activates only the concepts the instruction requires. A fixed instruction parser returns the required concepts $S(\ell, \mathcal{L})$, and `DepClosure` adds the concepts they depend on. For each active concept, `Inst` extracts its parameters η_c from the instruction:

$$\mathcal{C}_\ell = \text{DepClosure}(S(\ell, \mathcal{L})), \quad \eta_c = \text{Inst}(\ell, c), \quad c \in \mathcal{C}_\ell. \quad (3)$$

The parameters η_c include entity phrases for *objects*, *destinations*, *controls*, and also the required properties, such as *counts*, *ordinals*, and *order constraints* (Appendix C). These two operations output answers to different questions: $\mathcal{C}_\ell$ decides which concepts to recognize, and η_c decides which entities each concept grounds, so each record binds to the object the task refers to rather than to any object in view. Task conditioning only decides what to observe but does not establish facts: *repeat three times* sets a desired count, and only observed completion events increase the completed count.

Multimodal concept recognition. No single modality recognizes a concept reliably. Vision finds objects but cannot tell whether the gripper holds one; robot state tells when the gripper closes but not on what. Each recognizer therefore composes evidence across modalities. We harness OWLv2 (Minderer et al., 2023) to ground the entity phrases as bounding boxes, and SAM2 (Ravi et al., 2025) to track them across frames where needed. End-effector positions and gripper states show when and where the robot acts, and camera calibration projects the gripper into the image, so visual and robot-state evidence can be compared directly.

Each active concept c runs its recognizer $\text{Rec}_c(h_t; \eta_c)$, which composes these signals at three levels. Spatial and relational checks describe how entities are arranged at one time. Event checks test changes over an interaction interval. Temporal operators assemble confirmed events into counts or ordered sequences. Each result keeps its entity bindings, evidence, and time.

Evidence merging and memory update. Recognition outputs are noisy and repetitive. The same event appears in many frames, and objects disappear behind occluders. The Writer merges the outputs of all active concepts and updates the previous memory:

$$m_t = \text{Update}\left(m_{t-1}, \{\text{Rec}_c(h_t; \eta_c)\}_{c \in \mathcal{C}_\ell}\right). \quad (4)$$

Update turns noisy outputs into stable memory with four rules. First, a state needs accumulated evidence before it is confirmed, and a missed detection is not evidence against it; an occluded object therefore keeps its last location. Second, an event is recorded once, when it is first confirmed; seeing it again adds nothing. Third, a count is the number of distinct recorded events. Fourth, progress compares counts with the goal, and an ordered step counts only after its prerequisites are complete. All updates use only observations up to the current decision. A demonstration given with the task can set entity bindings and the required order, but its events never count as the robot’s own progress.

4.3 Integrating Concept Memory with VLA

The Reader R_ϕ turns the memory m_t into tokens u_t that the VLA π_θ can attend to.

Reading memory records as tokens. Each record $r \in m_t$ is a tuple of K discrete fields: kind, predicate, subject and object entities, truth value, status, confidence, age, progress flags, and grid location (Appendix F). Writing records as text would flatten this structure and lengthen the prompt, so the Reader embeds the fields directly. Each field k has its own embedding table E_k , because the same value means different things in different fields. Summing the field embeddings gives one vector $e(r)$ per record. A small Transformer T_ϕ then relates the records and maps each one to the VLA’s token width:

$$e(r) = \sum_{k=1}^K E_k(r_k), \quad u_t = R_\phi(m_t) = T_\phi(\{e(r)\}_{r \in m_t}). \quad (5)$$

The Reader’s parameters ϕ are the tables E_k and the Transformer T_ϕ . The Reader outputs one token per record, without pooling, so no fact is averaged away. Attention across records lets it relate facts, such as a count and its goal. Records form a set, so the Reader uses no position embeddings; temporal information enters through the age field and goal dependencies.

Policy integration and training. The tokens u_t join the image and language tokens in the prefix of π_θ , which we initialize from pretrained $\pi_{0.5}$ (Physical Intelligence et al., 2025). The pretrained policy has never seen memory tokens, so it must learn how they relate to actions. We therefore train ϕ jointly with the language backbone and action expert of π_θ , using the base model’s flow-matching objective. The vision encoder and the Writer W stay frozen. We compute training memory as at execution: m_t uses only observations up to decision t . Entity identifiers are arbitrary labels, so we randomly permute them in each training sample; the policy cannot rely on specific numbers. At execution, W updates m_t as observations arrive, and R_ϕ reads it before each action chunk.

Table 1: Success rates (%) on the 16 RoboMME memory-dependent tasks. AVG is the task-equal mean across all tasks. Baseline and human results are reported by RoboMME (Dai et al., 2026).

Method	Counting				Permanence				Reference				Imitation				AVG
	Bin Fill	Pick Xtimes	Swing Xtimes	Stop Cube	Video Umsk	Button Umsk	Video UmskS	Button UmskS	Pick HighL	Video Repick	Video PicBtn	Video PicOrd	Move Cube	Insert Peg	Pattern Lock	Route Stick	
Human Performance	96.00	100.00	80.00	78.00	90.00	92.00	92.00	90.00	92.00	92.00	98.00	90.00	90.00	98.00	84.00	86.00	90.50
FrameSamp+Modul	39.56	87.33	92.00	42.00	32.67	25.11	24.44	18.22	22.89	30.44	60.00	32.00	77.78	7.56	53.56	66.67	44.51
$\pi_{0.5}$	30.00	42.89	35.56	6.67	20.44	22.22	18.67	6.67	11.33	0.44	31.11	25.78	26.00	1.56	2.89	4.67	17.93
$\pi_{0.5}$ w/ past actions	26.67	58.33	26.67	4.67	30.67	23.67	20.67	16.00	12.33	8.67	24.00	18.67	34.00	1.00	4.00	5.67	19.73
SAM2Act+	40.00	76.00	25.33	0.00	27.33	32.00	18.00	26.67	17.33	5.33	24.67	20.00	29.33	0.00	0.00	0.00	21.37
MemER	56.67	79.33	59.33	0.00	81.33	72.00	38.00	21.33	70.67	25.33	30.00	26.00	82.67	6.67	16.67	12.00	42.38
ECoMEM	71.33	100.00	99.33	66.00	100.00	96.67	78.00	84.00	62.67	77.33	96.00	86.00	96.00	40.67	79.33	85.33	82.42

5 Experiments

Our experiments address four questions. **(1)** How does explicit concept memory compare with no-memory, history-based, and visual-memory alternatives (Section 5.2)? **(2)** Do different concept families provide distinct information needed for memory-dependent decisions (Section 5.3)? **(3)** What is required to construct useful concept memory, and how important are structured recognition, task-conditioned selection, and language grounding (Section 5.4)? **(4)** Can the concept library transfer to new real-robot tasks through reuse and extension (Section 5.5)?

5.1 Experiment Setup

Tasks. We evaluate on the 16 memory-dependent tasks of RoboMME (Dai et al., 2026) and two new real-robot tasks. RoboMME covers four memory demands: counting repeated events, maintaining objects through occlusion, referring to past interactions, and reproducing demonstrated behavior. On the real robot, **CupSwitch** requires completing a requested number of cup round trips between two plates, while **ScoopPour** requires a requested number of successful scoop-and-pour cycles; empty scoops do not count. These tasks introduce new objects and success conditions, and ScoopPour additionally requires a concept absent from the base library. We use a 7-DoF YAM arm and collect demonstrations with GELLO (Wu et al., 2024); further platform details are in Appendix I.

Baselines. On RoboMME, we compare against the baselines reported by RoboMME: $\pi_{0.5}$ without memory, $\pi_{0.5}$ with past actions, FrameSamp+Modul, SAM2Act+, and MemER. Together they cover no memory, history conditioning, and visual-memory approaches. We additionally compare the Writer with a fine-tuned VLM that extracts the same concepts directly from video (Section 5.4). On the real robot, we compare against $\pi_{0.5}$ fine-tuned on the same demonstrations without memory.

Training and evaluation. We initialize ECoMEM from pretrained $\pi_{0.5}$ and jointly train on all 16 RoboMME tasks for 200,000 updates. Each task is evaluated on 50 validation episodes with three policy sampling seeds, and we report per-task success and the task-equal mean. Controlled ablations use matched 50-episode evaluations and their own full-memory controls; their absolute scores are therefore not directly comparable with Table 1 (Appendices A and B). For the real robot, both methods are fine-tuned from pretrained $\pi_{0.5}$ on the same teleoperated demonstrations, and success requires completing exactly the requested number of cycles (Appendix I).

5.2 Concept Memory Outperforms Implicit and Visual Memory

Explicit concept memory substantially improves memory-dependent control. ECoMEM achieves 82.42% average success, 37.91 points above the strongest baseline, FrameSamp+Modul, and leads the evaluated policies on 15 of 16 tasks (Table 1). Its average is within 8.08 points of human performance.

The advantage is largest when the required information must persist beyond the visible frame. The largest margins occur on ButtonUnmaskSwap, VideoPlaceOrder, and VideoRepick, where the robot must act on a target, location, or ordering established earlier but no longer directly observable. Here, ECoMEM preserves entity bindings, locations, and event order as persistent records rather than requiring the policy to recover them from visual history. Counting tasks similarly benefit from explicit completion events and progress. By contrast, gains are smaller on SwingXtimes and MoveCube, where sampled or retrieved frames already expose much of the relevant history. PickHighlight is the only task where ECoMEM does not lead: MemER is 8 points higher, consistent with a transient visual cue being directly preserved in a retrieved keyframe.

Remaining errors expose control limits beyond memory. On InsertPeg, ECoMEM reaches 40.67%, compared with 7.56% for the strongest baseline, but remains well below human performance. Memory provides the insertion side and endpoint; precise execution must still be learned by the policy. Section 5.3 next isolates which information in concept memory drives these gains.

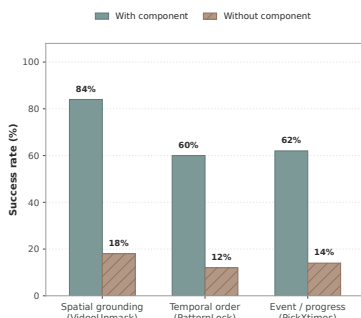


Figure 4: Different memory decisions depend on complementary concept families. Removing spatial grounding, temporal order, or event and progress information sharply reduces success on the corresponding memory-dependent task. Each pair is trained and evaluated with the same intervention on 50 matched episodes.

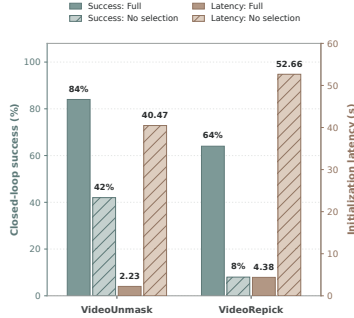


Figure 5: Task-conditioned concept selection improves both control and efficiency. With the policy fixed, full-library recognition adds task-unselected records, reduces success on VideoUnmask and VideoRepick, and increases memory initialization latency by 12.0 $\times$ to 18.1 $\times$. Latency is measured on matched episodes using the same GPUs.

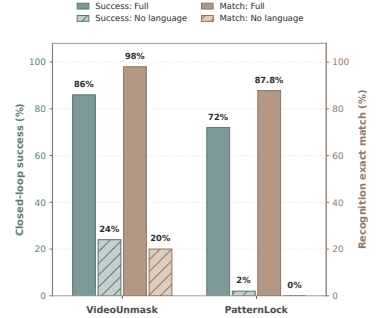


Figure 6: Language-conditioned grounding binds memory to the correct entities. Replacing instruction-specific entity phrases with generic role-level queries sharply reduces both reference-record agreement and closed-loop success on VideoUnmask and PatternLock. Concept selection and the policy are fixed.

5.3 Concept Families Provide Complementary Memory

A compact library is shared across diverse memory tasks. As shown in Figure 3, the 16 RoboMME tasks reuse concepts from the same four families rather than requiring task-specific memory representations. Tasks may differ in which concepts they activate and how they compose them.

Different memory decisions depend on different concept families. We remove spatial information on

Table 2: Concept extraction across all four memory families. We evaluate eight whole-video concepts on 95 RoboMME test videos. Structured and QA are separately fine-tuned Qwen3-VL-8B-Instruct models; Concept Memory uses our evidence-based Writer. Cell shading encodes extraction accuracy (darker is higher); **bold** denotes the best result in each row. See Appendix H for protocols and limitations.

Family	Concept extracted	Structured	QA	Concept Memory
Entity & Spatial	Picked-object identity	71.4%	71.4%	100.0%
	Highlighted-object set	63.2%	63.2%	97.4%
State & Relation	Currently highlighted	100.0%	97.4%	97.4%
	Currently holding queried object	100.0%	96.7%	100.0%
Event & Progress	Pick-place cycle count	86.7%	93.3%	100.0%
	Region-entry count	95.0%	90.0%	95.0%
Temporal & Procedure	Ordered object-destination sequence	11.8%	5.9%	100.0%
	Placements before/after press	23.5%	23.5%	88.2%

VideoUnmask, event and progress information on PickXtimes, and temporal order on PatternLock, while preserving the remaining memory and removing indirect cues to the ablated information (Appendix G.1). Each intervention is applied during both training and evaluation on matched data, allowing the policy to exploit any information that remains. Success drops from 84% to 18%, 62% to 14%, and 60% to 12%, respectively (Figure 4; exact McNemar $p < 0.002$ for all three). The failures align with the role of each family: *where* to act after occlusion, *how many* events have occurred, and *in what order* to act. Together, the coverage in Figure 3 and these targeted ablations show that a small shared vocabulary can support diverse memory demands while preserving the distinctions needed for control.

5.4 Reliable Memory Requires Structured Construction

The Writer must do more than preserve history: it must extract the right structure, select what is relevant, and bind records to the right entities. We test each requirement against a simpler alternative. For structured extraction, we compare Concept Memory with fine-tuned VLMs on concepts spanning all four memory families. For selection and grounding, we intervene on the Writer at evaluation while keeping the trained policy fixed and using matched full-memory controls.

Direct VLM extraction is strong on local state, but weak on structured history. We fine-tune Qwen3-VL-8B-Instruct to extract each concept directly from video, either as a structured record or as an answer to a question, and evaluate eight concepts on 95 held-out RoboMME videos (Table 2). The VLMs recover current state and simple event counts reliably: state-and-relation concepts reach 96.7–100%, and event counts reach 86.7–95.0%. Performance drops for episodic entity information (63.2–71.4%) and collapses on temporal composition, reaching only 5.9–11.8% for ordered object-destination sequences and 23.5% for placements around an event. The similar Structured and QA results indicate that output format is not the main bottleneck. In contrast, Concept Memory achieves 95–100% on the first three families and 88.2–100% on temporal concepts. Explicitly composing grounded evidence into events and temporal records therefore provides a substantially more reliable memory construction mechanism than the tested direct video extraction.

Task-conditioned selection improves both accuracy and efficiency. *No Selection* runs every recognizer and adds all resulting records while retaining the task-relevant ones. Success drops from 84% to 42% on

Table 3: Real-robot success (successes/trials) by requested number of cycles; success requires exactly that count. Pooled uses the targets both policies were run on.

Method	ScoopPour			CupSwitch		Pooled	
	1×	2×	3×	2×	3×	succ./trials	rate
ECoMEM	16/20	19/23	11/12	11/12	11/12	68/79	86.1%
$\pi_{0.5}$ (no memory)	4/12	0/12	0/11	0/12	1/11	5/58	8.6%

CupSwitch (reuse: base concepts only)

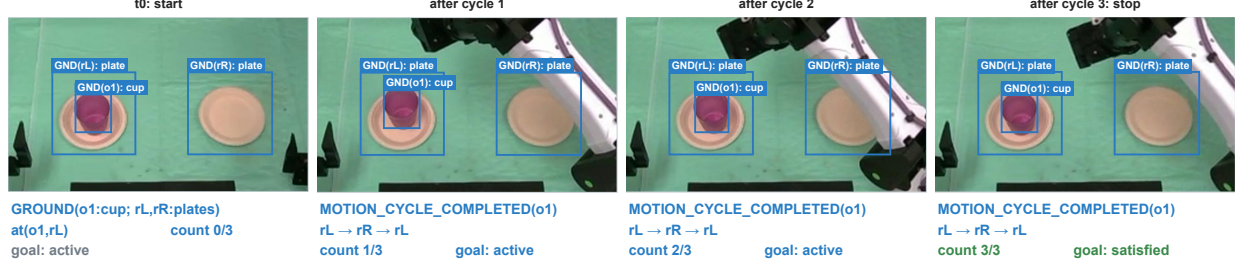


Figure 7: Concept memory transfers through reuse and compositional extension on a real robot. **Blue:** reused base concepts; **orange:** the new SCOOP concept. Both tasks require counting, but ECoMEM represents progress at the task-relevant granularity: CupSwitch advances after each completed round trip, while ScoopPour composes SCOOP with the same counter so that only successful scoops carrying beans advance progress. An empty scoop therefore leaves the count unchanged and automatically triggers a retry. Records are produced online by the Writer.

VideoUnmask and from 64% to 8% on VideoRepick, while median memory initialization becomes $18.1\times$ and $12.0\times$ slower (Figure 5). Thus, more memory is not necessarily better: task-conditioned selection reduces recognition cost and prevents irrelevant records from disrupting the policy.

Without language grounding, records bind to the wrong objects. *Generic queries* replace task-specific entity phrases with role-level ones such as *an object*, while keeping the selected concepts, detection budget, and policy instruction fixed. The Writer runs the right concepts but no longer knows which entities they refer to. Agreement with reference records falls from 98% to 20% for target locations in VideoUnmask and from 87.8% to 0% for ordered location sequences in PatternLock; closed-loop success falls from 86% to 24% and from 72% to 2% (Figure 6). The policy still receives the full instruction, yet it cannot repair memory records grounded to the wrong objects.

Together, these results show that useful memory depends on how it is constructed. Reliable memory requires structured evidence composition, task-conditioned selection, and language-grounded entity binding; the policy cannot reliably recover information that the Writer constructs incorrectly.

5.5 The Library Transfers to New Tasks on a Real Robot

The real robot tests whether the library transfers beyond RoboMME to new cameras, objects, and tasks. We consider two forms of transfer: **CupSwitch** tests whether a new task can reuse the existing library, while **ScoopPour** tests whether a missing capability can be added compositionally. Both policies initialize from pretrained $\pi_{0.5}$ and use the same real-robot demonstrations; ECoMEM additionally receives concept memory.

Transfer occurs through reuse and local extension. CupSwitch uses only existing concepts for grounding, persistence, pick-place cycles, event counting, and count goals, with no new recognizer or update rule (Figure 3). ScoopPour adds a single concept, SCOOP(tool, material), which confirms a scoop only when the lifted scooper carries beans. Grounding, persistence, counting, and goal tracking are reused unchanged. Thus, one new task is covered directly, while the other extends the library with one concept that composes with the existing machinery.

Conditioning on explicit memory is critical for repeated tasks. ECoMEM succeeds in 80% to 92% of ScoopPour trials and 92% of CupSwitch trials, remaining high across requested counts (Table 3). Without memory, $\pi_{0.5}$ reaches 33% on one ScoopPour cycle but 0% on two and three, and reaches 0% and 9% on two and three CupSwitch round trips. Across shared evaluation conditions, ECoMEM succeeds in 68/79 trials (86.1%), compared with 5/58 (8.6%) for the no-memory policy (Fisher exact $p < 10^{-20}$; trials are unpaired and unequal in number). The two systems share the pretrained backbone and demonstrations, while only ECoMEM conditions the policy on explicit memory tokens. The collapse of the no-memory policy as counts increase shows that current observations alone are insufficient once task progress is no longer visible.

Neural-symbolic concepts let memory adapt its granularity to the task. Because concepts compose learned perceptual recognition with explicit event and progress structure, the same memory machinery can track different notions of task progress. Figure 7 shows this online. In CupSwitch, the counter advances after each completed round trip, distinguishing visually similar scenes by how many cycles have occurred. In ScoopPour, SCOOP adds a success condition, so only scoops that actually carry beans advance the same counter; an empty scoop leaves progress unchanged and the policy retries. Thus, the library can represent progress at the semantic granularity each task requires by composing shared counting machinery with task-relevant concepts.

6 Conclusion

We introduced ECoMEM, an explicit concept memory interface that separates evidence-based memory construction from learned action generation. A shared, compositional concept library represents task-relevant world state, events, progress, and temporal structure, while the Writer maintains these records and the Reader learns how to use them for control. Across 16 RoboMME tasks, ECoMEM substantially outperforms no-memory, history-based, and visual-memory alternatives. Controlled ablations show that distinct concept families provide complementary information, and that structured recognition, task-conditioned selection, and language grounding are critical to constructing effective memory. On new real-robot tasks, the same library transfers through direct reuse or the addition of a single task-specific concept, showing that explicit concepts can provide a reusable and extensible memory interface for VLA control.

References

- Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. AriGraph: Learning knowledge graph world models with episodic memory for LLM agents. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, 2025. doi: 10.24963/ijcai.2025/2.
- Abraar Anwar, John Welsh, Joydeep Biswas, Soha Pouya, and Yan Chang. ReMEmbR: Building and reasoning over long-horizon spatio-temporal memory for robot navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL <https://arxiv.org/abs/2409.13682>.
- Ashay Athalye, Nishanth Kumar, Tom Silver, Yichao Liang, Jiuguang Wang, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. From pixels to predicates: Learning symbolic world models via pretrained VLMs. *IEEE Robotics and Automation Letters*, 11(4):4002–4009, 2026. URL <https://ieeexplore.ieee.org/document/11373861>.
- Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2603.04639>.
- Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 15925–15942, 2025. URL <https://proceedings.mlr.press/v267/fang25c.html>.
- Qiao Gu, Alihusein Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5021–5028, 2024. URL <https://concept-graphs.github.io/>.
- Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782, 2023. URL <https://proceedings.mlr.press/v205/huang23c.html>.
- Vidhi Jain, Maria Attarian, Nikhil J Joshi, Ayzaan Wahid, Danny Driess, Quan Vuong, Pannag R Sanketi, Pierre Sermanet, Stefan Welker, Christine Chan, Igor Gilitschenski, Yonatan Bisk, and Debidatta Dwibedi. Vid2Robot: End-to-end video-conditioned policy learning with cross-attention transformers. In *Robotics: Science and Systems*, 2024. URL <https://arxiv.org/abs/2403.12943>.

- Yunfan Jiang, Yevgen Chebotar, Ruijie Zheng, Fengyuan Hu, Yunhao Ge, Jimmy Wu, Tianyuan Dai, Scott Reed, Li Fei-Fei, Yuke Zhu, and Linxi Fan. RoboTTT: Context scaling for robot policies. *arXiv preprint arXiv:2607.15275*, 2026.
- Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. HAMLET: Switch your vision-language-action model into a history-aware policy. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2510.00695>.
- Hang Li, Fengyi Shen, Dong Chen, Liudi Yang, Xudong Wang, Jinkui Shi, Zhenshan Bing, Ziyuan Liu, and Alois Knoll. ReMem-VLA: Empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv preprint arXiv:2603.12942*, 2026.
- Yichao Liang, Nishanth Kumar, Hao Tang, Adrian Weller, Joshua B. Tenenbaum, Tom Silver, João F. Henriques, and Kevin Ellis. VisualPredicator: Learning abstract world models with neuro-symbolic predicates for robot planning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.23156>.
- Peiqi Liu, Zhanqiu Guo, Mohit Warke, Soumith Chintala, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. DynaMem: Online dynamic spatio-semantic memory for open world mobile manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL <https://arxiv.org/abs/2411.04999>.
- Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3468–3484, 2023. URL <https://proceedings.mlr.press/v229/liu23g.html>.
- Pierrick Lorang, Johannes Huemer, Timothy Duggan, Kai Goebel, Patrik Zips, and Matthias Scheutz. Build on priors: Vision–language–guided neuro-symbolic imitation learning for data-efficient real-world robot manipulation. *arXiv preprint arXiv:2604.03759*, 2026.
- Jiayuan Mao, Chuang Gan, Pushmeet Kohli, Joshua B. Tenenbaum, and Jiajun Wu. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In *International Conference on Learning Representations*, 2019. URL <https://nscl.csail.mit.edu/>.
- Jiayuan Mao, Tomás Lozano-Pérez, Joshua B. Tenenbaum, and Leslie Pack Kaelbling. PDSketch: Integrated domain programming, learning, and planning. In *Advances in Neural Information Processing Systems*, volume 35, pages 36972–36984, 2022.
- Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. STRAP: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2412.15182>.
- Matthias Minderer, Alexey Gritsenko, and Neil Houlsby. Scaling open-vocabulary object detection. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/e6d58fc68c0f3c36ae6e0e64478a69c0-Abstract-Conference.html.

- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL <https://arxiv.org/abs/2504.16054>.
- Jianing Qi, Jiawei Liu, Hao Tang, and Zhigang Zhu. Beyond semantics: Rediscovering spatial awareness in vision-language models. *arXiv preprint arXiv:2503.17349*, 2025.
- Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 23–72. PMLR, 2023. URL <https://proceedings.mlr.press/v229/rana23a.html>.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=Ha6RTeWMd0>.
- Mohammad Reza Samsami, Artem Zhohus, Janarthanan Rajendran, and Sarath Chandar. Mastering memory tasks with world models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2403.04253>.
- Ashish Seth, Utkarsh Tyagi, Ramaneswaran Selvakumar, Nishit Anand, Sonal Kumar, Sreyan Ghosh, Ramani Duraiswami, Chirag Agarwal, and Dinesh Manocha. EGOILLUSION: Benchmarking hallucinations in egocentric video understanding. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 28461–28480, 2025. URL <https://aclanthology.org/2025.emnlp-main.1446/>.
- Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2508.19236>.
- Tom Silver, Rohan Chitnis, Nishanth Kumar, Willie McClinton, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Joshua B. Tenenbaum. Predicate invention for bilevel planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 12120–12129, 2023. doi: 10.1609/aaai.v37i10.26429.
- Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Scaling up memory for robotic control via experience retrieval. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=1dH4ARGdwD>.
- Moritz Tenorth and Michael Beetz. KnowRob: Knowledge processing for autonomous personal robots. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 4261–

- 4266, 2009. doi: 10.1109/IROS.2009.5354602. URL <https://ai.uni-bremen.de/papers/tenorth09knowledge.pdf>.
- Marcel Torne, Andy Tang, Yuejiang Liu, and Chelsea Finn. Learning long-context diffusion policies via past-token prediction. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 1744–1755. PMLR, 2025. URL <https://proceedings.mlr.press/v305/villasevil25a.html>.
- Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. DayDreamer: World models for physical robot learning. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 2226–2240. PMLR, 2023.
- Philipp Wu, Yide Shentu, Zhongke Yi, Xingyu Lin, and Pieter Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12156–12163. IEEE, 2024. doi: 10.1109/IROS58592.2024.10801581.
- Yiqing Xu, Jiayuan Mao, Yilun Du, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and David Hsu. Set It Up!: Functional object arrangement with compositional generative models. In *Robotics: Science and Systems*, Delft, Netherlands, 2024. doi: 10.15607/RSS.2024.XX.095.
- Yiqing Xu, Linfeng Li, Cunjun Yu, and David Hsu. “Stack It Up!”: 3D stable structure generation from 2D hand-drawn sketch. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 124–151. PMLR, 2025.
- Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, Wengang Zhou, Yao Mu, and Tai Wang. EventVLA: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv preprint arXiv:2606.20092*, 2026.
- Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R. Walter. Statler: State-maintaining language models for embodied reasoning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15083–15091, 2024. doi: 10.1109/ICRA57147.2024.10610634.
- Juntu Zhao, Wenbo Lu, Di Zhang, Yufeng Liu, Yushen Liang, Tianluo Zhang, Yifeng Cao, Junyuan Xie, Yingdong Hu, Shengjie Wang, Junliang Guo, Dequan Wang, and Yang Gao. Do you need proprioceptive states in visuomotor policies? *arXiv preprint arXiv:2509.18644*, 2025.

A Policy Training

A.1 Shared policy configuration.

All policies use pretrained $\pi_{0.5}$ [Physical Intelligence et al. \(2025\)](#), a 2B language backbone, and a 300M action expert. The image backbone is frozen; the language backbone, action expert, and memory reader are optimized jointly. Recognition and memory writing remain fixed. All policy training is performed on NVIDIA H200 GPUs. The ECoMEM policy predict 20 actions and execute the first 16 before requesting another chunk.

A.2 Main benchmark training.

We initialize ECoMEM from pretrained $\pi_{0.5}$ and jointly train on all 16 RoboMME tasks. Samples are drawn uniformly from the retained observation–action frames. Reported results use a single EMA checkpoint. The training process takes $4 \times$ H200 GPU for 24 hours.

A.3 Controlled comparisons.

Each content-ablation pair is trained independently from the same pretrained $\pi_{0.5}$ policy. The intervention is applied identically in training and evaluation. Each pair has identical observation/action indices and normalization. Each experiment retains its own matched full-memory control. Training budgets and memory-construction settings differ across the main benchmark and the controlled comparisons.

A.4 Flow-matching objective.

Let $\mathcal{D}$ contain examples (ℓ, o_t, m_t, a_t^*) , with instruction ℓ , current observation o_t , causal memory m_t , and expert action chunk a_t^* at decision t (Section 3). For each sample, σ is a uniformly sampled bijection of its active entity identifiers, applied consistently to both subject and object fields. Identifier 0 is fixed. This renames entities, not record positions. We sample $b \sim \text{Beta}(1.5, 1)$ and set $\tau = 0.999b + 0.001$; independently, ϵ is standard Gaussian noise with the action chunk’s shape. Writing R_ϕ for the Reader and v_θ for the policy’s conditional velocity predictor gives

$$\begin{aligned} a_t^{(\tau)} &= (1 - \tau)a_t^* + \tau\epsilon, \\ \hat{v}_t &= v_\theta\left(a_t^{(\tau)}, \tau \mid \ell, o_t, R_\phi(\sigma(m_t))\right). \end{aligned} \tag{6}$$

The optimized parameters are the Reader weights ϕ and the unfrozen policy weights θ :

$$\min_{\phi, \theta} \mathbb{E}_{\mathcal{D}, \sigma, b, \epsilon} \left[\|\hat{v}_t - (\epsilon - a_t^*)\|_2^2 \right]. \tag{7}$$

The squared error is averaged over action-chunk entries.

B Main Benchmark Evaluation

Baseline scores are taken from Table 3 of the first public RoboMME manuscript [Dai et al. \(2026\)](#). The descriptions below summarize the corresponding methods and benchmark adaptations.

B.1 Current-observation and past-action baselines.

The standard $\pi_{0.5}$ conditions action generation on the current images, robot state, and task instruction, without historical visual inputs. The past-action variant additionally concatenates previous actions with the language tokens. Its historical information therefore describes issued actions rather than the visual outcomes of those actions.

B.2 Symbolic memory: SimpleSG and GroundSG.

An auxiliary VLM predicts the next subgoal from the current image and accumulated subgoal history. SimpleSG expresses this subgoal as a language command; GroundSG additionally includes image coordinates to identify the intended target. The predicted subgoal is appended to the task instruction for $\pi_{0.5}$ to execute. RoboMME evaluates prompted Gemini-2.5-Pro, Qwen3-VL-4B fine-tuned on subgoal annotations, and simulator-provided oracle subgoals.

B.3 Perceptual memory: FrameSamp and TokenDrop.

Both methods retain historical features extracted by the vision encoder. FrameSamp uniformly samples frames and spatially pools their visual tokens. TokenDrop instead ranks image patches by temporal RGB differences and retains informative patches under a fixed token budget. It preserves the first frame and uses spatiotemporal positional encoding. Thus, FrameSamp selects whole observations, whereas TokenDrop selects changing regions across observations.

B.4 Recurrent memory: RMT and TTT.

These methods summarize incoming visual features through recurrent updates rather than retaining a growing collection of past frames. RMT maintains a fixed set of memory slots. At each update, attention jointly processes the previous slots and a chunk of incoming tokens; the updated slot representations are carried to the next chunk and subsequently supplied to the policy.

TTT instead stores historical information in the fast weights of a small learned module. Incoming features produce training targets for a local self-supervised update, after which the updated weights transform query features into history-conditioned outputs. The fast weights are carried across chunks, while the resulting features provide the memory representation used for action generation. RMT therefore updates a token-valued state, whereas TTT updates a parameter-valued state.

B.5 Memory integration.

Each perceptual or recurrent representation is evaluated with three integration mechanisms. *Context* places memory tokens alongside the observation and instruction tokens for processing by the VLM. *Modul* lets action features query memory through cross-attention; the retrieved features generate scale and shift parameters that modulate normalized action features before the feed-forward layers. *Expert* processes memory through a separately parameterized transformer branch. The action expert attends to both this branch and the VLM branch, while memory and VLM features remain separate. These variants distinguish joint input processing, feature modulation, and a dedicated memory-processing pathway.

B.6 MemER.

MemER [Sridhar et al. \(2026\)](#) uses a hierarchical policy with persistent keyframe memory. A high-level VLM selects task-relevant historical frames and combines them with recent observations to generate language subtasks. A low-level VLA executes these subtasks using current sensory inputs. Keyframe filtering consolidates repeated selections, retaining salient observations across long interactions. RoboMME adapts MemER using fine-tuned Qwen3-VL-4B and the GroundSG policy.

B.7 SAM2Act+.

SAM2Act+ [Fang et al. \(2025\)](#) augments a multi-view manipulation policy with a memory encoder, memory bank, and memory attention. The encoder stores visual features together with spatial action information; memory attention retrieves this context when predicting subsequent actions. This supports recalling previously observed locations and interactions. In RoboMME, predicted keyframe actions are executed by a motion planner, and demonstration videos prefill the memory bank.

C Concept Library

Complete library organized by semantic family. Table 4 specifies the remembered concepts and their representations. It covers all named base predicate, event, and role codes, together with entity bindings, spatial fields, counts, and temporal structures. Padding and unknown-value codes are encoding sentinels, not concepts. Arguments below name semantic participants rather than serialization columns: entity, gripper, container, region, control, or goal. Each instance uses episode-local entity references; confidence, evidence, and time accompany the applicable records as specified in Appendix F.

Table 4: Semantic inventory of the base Concept Memory interface, grouped into four families. Monospaced names are record codes; italicized entries are fields or derived structures. Predicate arguments name the subject and, where applicable, the object. Gripper identity is stored separately as agent metadata.

Concept / arguments	Meaning and representation
Entity & Spatial Grounding	
<i>Entity identity and language binding</i>	Episode-local entity keys link observations, relations, and events. Groundings retain instruction phrases and aliases; the policy interface encodes entity slots rather than alias strings.
<code>role:cube, role:container, role:object, role:region, role:robot</code>	The five grounding-role codes in the base policy vocabulary. Each grounding records an entity key, role, aliases, confidence, and source.
<i>Entity location and endpoint anchor</i>	Grounded entity or tool-endpoint locations are supplied to the Reader as row and column cells on a 32×32 image grid. Cell indices are 1-based; zero denotes an unspecified position.
<code>occupies_region(entity, region)</code>	Selected goal records also carry the subject’s target cell. Current occupancy of a grounded region. Entry and exit are represented by separate transition events.
<code>side:left, side:right</code>	Entity-bound categorical insertion-side predicates, distinct from the demonstrated endpoint’s spatial anchor.
State & Relation	
<code>gripper_contact(entity)</code>	Supported contact with the gripper. The interaction writer stores the gripper in agent metadata, not as the predicate’s object.

Table 4 continued

Concept / arguments	Meaning and representation
attached(entity)	Supported attachment to the gripper, with independent physical evidence beyond gripper closure.
holding(entity)	Supported control of an entity by the gripper; lifting is represented separately.
lifted(entity)	The entity has risen from its support.
object_motion(entity)	Supported object motion, including transport during interaction.
inside(entity, container)	Containment relative to a grounded container.
visible(entity)	Supported visibility. Missing evidence alone does not establish a confirmed negative state.
activated(control)	Activation state of a grounded button or control.
inserted(entity, container)	Insertion relation between a manipulated entity and its destination.
release(entity, destination)	Supported release; the destination is stored when provided.
manner:peg_push, manner:gripper_push, manner:grasp_putdown	Entity-bound categorical predicates for demonstrated tool-mediated push, gripper push, or grasp-and-put-down.
Event & Progress	
CONTACT_ACQUIRED, CONTACT_RELEASED	Confirmed positive and negative transitions of gripper_contact.
ATTACHMENT_ESTABLISHED, ATTACHMENT_CLEARED	Confirmed positive and negative transitions of attached.
HOLDING_ESTABLISHED, HOLDING_CLEARED	Confirmed positive and negative transitions of holding.
TRANSPORT_STARTED, TRANSPORT_ENDED	Confirmed positive and negative transitions of object_motion.
CONTAINMENT_ESTABLISHED, CONTAINMENT_CLEARED	Confirmed positive and negative transitions of inside.
CONTROL_ACTIVATED	A newly confirmed positive activated state.
INSERTION_ESTABLISHED, INSERTION_CLEARED	Confirmed positive and negative transitions of inserted.
REGION_ENTERED, REGION_EXITED	Confirmed positive and negative transitions of occupies_region.
RELEASE_CONFIRMED	A newly confirmed positive release state.
VISIBILITY_ACQUIRED, VISIBILITY_LOST	Confirmed positive and negative transitions of visible.
motion_cycle_complete(unit, label), MOTION_CYCLE_COMPLETED	Completion predicate and derived event for a distinct recognized unit. The progress writer creates one unit entity per completion; its object binding identifies the operation or category.
procedure_milestone_complete(subject, object), PROCEDURE_MILESTONE_COMPLETED	Completion predicate and derived event for a procedural milestone; the object binding is optional.
<i>Observed repetition count</i>	Distinct completed-unit records encode accumulated progress. The field-based Reader receives these records, not a separate scalar count column; category-specific counts follow from their object bindings.
<i>Goal satisfaction</i>	Goal records specify desired grounded predicates. Progress marks goals as satisfied, executable frontier, or blocked. Required repetition counts are task parameters, separate from observed completions.

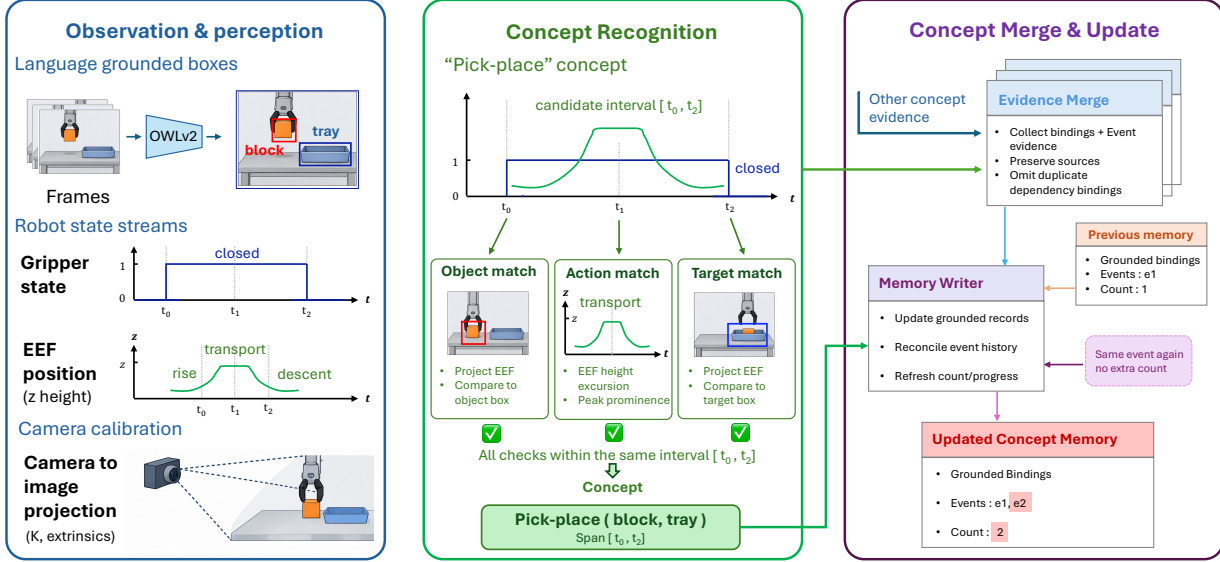


Figure 8: Concept recognition and memory update in ECoMEM. A pick-and-place interaction illustrates how language-grounded entities and robot-state evidence jointly establish an event. The writer merges concept evidence, updates grounded records, and advances progress once per distinct completion.

Table 4 continued	
Concept / arguments	Meaning and representation
Temporal & Procedure	
<i>Event time and evidence provenance</i>	Event records store occurrence time, evidence IDs, source, and confidence. The Reader receives a quantized relative age rather than an absolute timestamp or evidence-ID string.
<i>Ordered sequence</i>	Ordered bindings, goal nodes, and serialized records preserve demonstrated entity, control, or waypoint sequences. Sequence is a composed structure, not a separate ORDERED_SEQUENCE token in the base field vocabulary.
<i>Ordinal and before/after reference</i>	Language parameters specify an ordinal, reference event, and temporal relation. Recognition resolves them into the corresponding observed entity or destination.
route:left_clockwise, route:left_counterclockwise, route:right_clockwise, route:right_counterclockwise	Categorical route predicates attached to ordered hop entities, with associated waypoint groundings. Route belief records retain hop order.
<i>Goal prerequisites and active step</i>	Goal nodes retain prerequisite IDs and current-state or ever-achieved satisfaction modes. The writer derives frontier and blocked flags; the field interface encodes these flags rather than prerequisite-ID lists.

D Observation & Perception

Figure 8 illustrates the recognition and update process through a pick-and-place interaction. Its left and middle panels show how shared perception outputs are composed into concept evidence; the resulting memory update is described in Appendix F.

D.1 Language-grounded perception.

A shared, frozen OWLv2 detector [Minderer et al. \(2023\)](#) grounds instruction-derived entity descriptions in camera observations. Detections are associated with episode-local entity identities, and their locations are normalized and quantized onto a shared spatial grid.

D.2 Temporal sampling and tracking.

Recognition processes the available observation prefix. Keyframe selection prioritizes gripper transitions and visual changes while maintaining temporal coverage. Tracking uses image motion, segmentation masks, and robot–object motion correspondence to update entity associations. During occlusion, memory retains the entity binding and its last-supported location. Camera observations and robot states are aligned along the execution timeline, including observations between policy queries. This aligned stream supplies the evidence used by the selected concept recognizers.

E Concept Recognition

E.1 Concept selection.

The concept library provides reusable recognition capabilities and their dependencies. For all the tasks that have been seen, the model will distill a small and fast router to select concepts. This method was applied in the RoboMME simulation. However, for tasks that have not been seen before, or for combinations of different tasks, we have introduced a new option to utilize the LLM agent to select possible concepts. The task instructions and library descriptions will guide an LLM to propose a Concept Plan: a set of concepts to track during execution. Dependency expansion adds the supporting concepts, and validation checks library membership, dependency consistency, and required sensor inputs. The instruction supplies entity descriptions, target counts, and temporal constraints. These arguments instantiate the selected recognizers. Selection specifies the concepts to track, and recognition evaluates them against sensor evidence.

E.2 Language-grounded concept recognition.

The selected recognizers combine visual evidence with end-effector motion, gripper state, and camera calibration to identify relations, interaction events, and ordered bindings. Each recognizer associates its result with grounded entities, supporting evidence, and an observation time or interaction interval. Spatial concepts describe entity configurations; event concepts identify changes over an interval; temporal concepts organize observed interactions.

E.3 Composing evidence over an interaction.

The middle panel of Figure 8 illustrates three complementary checks. Gripper closure at t_0 and opening at t_2 delimit a candidate interaction interval $[t_0, t_2]$. *Object match* projects the end-effector position at acquisition into the image and compares it with the grounded object’s box. *Action match* evaluates the candidate lift-and-transport motion using the end-effector’s vertical excursion and height peak above the interval endpoints. *Target match* compares the projected release position with the grounded destination’s box. Camera intrinsics and extrinsics provide the common image-coordinate system for these checks. Object and destination matching use detections available by their respective interaction times.

When all checks hold for the same interval, the recognizer produces a completed pick-and-place instance with its object, destination, and temporal span. In the figure, these bindings are the block and tray. Each concept specifies the entity, physical, and temporal evidence used in its recognition rule.

F Concept Merge & Update

The right panel of Figure 8 shows how recognized concepts update a persistent memory. Evidence Merge combines the selected recognizers’ outputs; the Memory Writer integrates them with previous records and refreshes event history and progress. The Reader then converts these records into memory tokens for the VLA policy.

F.1 Evidence merging.

Each recognizer produces entity bindings and supporting observations with their sources and temporal references. Evidence Merge collects these outputs and retains their provenance. Shared dependency evidence is associated with its composite result, avoiding duplicate bindings for intermediate stages. The merged evidence supplies the writer with the selected concepts’ spatial, relational, and temporal outputs.

F.2 State estimation and commitment.

For a grounded predicate identified by its predicate, subject, optional object, and agent, the writer accumulates supporting and opposing evidence over time. Let L_t be its log-odds at update t , initialized to zero. Channel j contributes a signed score $s_{j,t} \in [-1, 1]$ with positive weight w_j . Let Δ_t denote elapsed seconds since the previous update, $h > 0$ the decay half-life, $B > 0$ the log-odds bound, and $\delta_t = \min(\max(\Delta_t, 10^{-9}), \delta_{\max})$ the effective evidence duration, capped by $\delta_{\max} > 0$. The update is

$$L_t = \text{clip}_{[-B, B]} \left(2^{-\Delta_t/h} L_{t-1} + \delta_t \sum_j w_j s_{j,t} \right), \quad (8)$$

where the clipping operator bounds its argument to $[-B, B]$. The first update uses $\Delta_t = 0$.

Separate confirmation and retention thresholds provide hysteresis: strong evidence establishes a state, while a lower level of support maintains it. Opposing evidence can establish a false state, and insufficient support returns the estimate to unknown. Attachment, holding, and lifting additionally require physical evidence such as tracked object motion or relative-pose consistency. Confirmed updates replace the previous state for the same grounded predicate. Mutually exclusive hypotheses, such as two objects held by one gripper, are resolved using their accumulated support, with inferred exclusions marked in the records.

F.3 Event history and progress.

The writer records newly confirmed predicate transitions and recognized interaction completions. Event history is reconciled across successive recognition updates using completion labels and temporal correspondence. Evidence matched to an existing completion updates that entry; a newly identified completion adds an event. Unchanged state observations retain the existing state; a later reversal and renewed transition can establish another event instance.

In Figure 8, the previous memory contains one completed interaction, e_1 . The newly recognized pick-and-place interaction contributes a second event, e_2 , increasing the count from one to two. Evidence matched

again to e_2 updates the existing entry and leaves the count unchanged. Completed units receive distinct bindings, so repetition progress is represented by the collection of completed-unit records and their operation or category associations.

Goal records specify desired grounded states and prerequisite goals. Current-state goals are evaluated against the committed state; persistent completion goals retain achieved progress. A dependent completion requires matching evidence after its prerequisites have been achieved. The writer updates satisfied, executable-frontier, and blocked flags accordingly. These records expose estimated world state, completed interactions, and goal progress to the policy.

F.4 Record interface.

Table 5 summarizes the eleven integer fields supplied to the Reader. Entity slots are episode-local, and references to the same entity share a slot across record types. Goals, beliefs, events, groundings, and geometry records are serialized in that order, retaining their within-type ordering. The memory store retains full timestamps and evidence provenance; the Reader uses the corresponding compact fields. Padding rows are excluded by a Boolean mask.

Table 5: Structured fields supplied to the memory Reader. Subject/object and row/column each denote two fields.

Field	Encoding
Kind	Record type: belief, event, goal, grounding, or geometry; zero denotes padding.
Predicate	Shared identifiers for predicates, transition events, entity roles, and categorical attributes.
Subject, object	Episode-local entity identifiers; zero denotes an unspecified argument.
Truth	True, false, or unknown; zero denotes an inapplicable value.
Status	Tentative, confirmed, or stale; zero denotes an inapplicable value.
Confidence	$\text{round}(9p)$ for confidence $p \in [0, 1]$.
Age	Zero for nonpositive age; otherwise $\min(15, 1 + \lfloor \log_2 d \rfloor)$, where d is the elapsed number of frames since the record timestamp.
Flags	Satisfied, executable frontier, blocked, and derived exclusion indicators.
Row, column	Cells 1–32 on a 32×32 image grid; zero denotes an unspecified position.

Grounding records carry entity locations, and selected goal records also carry their subject’s target location. Relative ages and ordered bindings encode temporal information, while completion records and goal flags expose progress to the policy.

F.5 Reader architecture.

Each field is mapped to a 256-dimensional embedding. The eleven embeddings are summed and layer-normalized to form one representation per record. Two pre-normalized Transformer blocks integrate information across valid records, using four attention heads and feed-forward width 1,024 with GELU activations. Output normalization and a linear projection produce one 2,048-dimensional token per record, scaled by $\sqrt{2048}$. These memory tokens join the image and language tokens in the VLA prefix. The Reader is trained jointly with the policy, allowing action learning to use the maintained concept records.

F.6 Causal training alignment.

At each decision frame, the training sample pairs the current observation and memory constructed through that frame with the subsequent expert action chunk. Demonstrated references and execution progress are

maintained separately: a demonstration specifies what to reproduce, while observed execution establishes what has been completed. An event enters a sample’s memory once its supporting evidence is available. Entity-identifier augmentation applies a shared bijection across all records in a sample, preserving the connections between groundings, relations, events, and goals.

G Ablations of pipeline

G.1 Content interventions.

Figure 4 modifies the record interface, with current images, robot inputs, action targets, and task requirements held fixed within each pair.

- **No Spatial (VideoUnmask):** set row and column fields to zero in every active record, including any goal or geometry row carrying coordinates. All other fields and masks are unchanged.
- **No Temporal (PatternLock):** preserve the active fact multiset and spatial cells, zero every age, consistently remap entity slots by grounding location, and lexicographically sort the resulting rows before packing the mask. Grounding ties use predicate and then original-slot tie-breaks. Progress flags and fact multiplicities remain. This intervention removes relative-age and ordered-binding cues from the record presentation.
- **No Event/Progress (PickXtimes):** remove event rows and beliefs encoding cycle or procedure completion, clear the satisfied, frontier, and blocked bits, remove groundings referenced only by deleted records, zero ages, canonicalize entity slots, and sort/pack the surviving rows. The positive condition exposes semantic cycle and procedure completion events while excluding low-level contact, holding, and transport event rows from the Reader. Both conditions retain the current-state evidence needed by recognition.

For an operational example, a grounding row (4, role:object, 2, 0, 0, 0, 9, 0, 0, 12, 21) becomes (4, role:object, 2, 0, 0, 0, 9, 0, 0, 0, 0) under No Spatial; the predicate name denotes its integer code. Under No Temporal, nonzero belief/event ages become 0, entity slots are consistently renamed, and records are re-sorted. Under No Event/Progress, completion events are removed, surviving records are packed contiguously, and goal-progress bits are cleared.

G.2 Initialization latency.

Validation episodes 0–4 are timed for each condition, sequentially on the same pair of RTX 3090 GPUs. The timer starts before the recognition call and ends after evidence merging, memory construction, and interface validation. The measured interval includes first-use detector loading and warmup occurring within these boundaries. VLA loading and action execution occur outside the timed interval. The median of five episodes is 2.23 s versus 40.47 s for VideoUnmask and 4.38 s versus 52.66 s for VideoRepick. Corresponding means are 3.87/41.41 s and 5.51/56.61 s. These measurements refer to per-episode initialization.

Table 6 collects these measurements. The ratios in the main text compare median per-episode initialization times: No Selection divided by task-conditioned selection.

Table 6: Memory initialization on matched episodes using the same pair of RTX 3090 GPUs. Times are in seconds; each condition uses five episodes. C denotes task-conditioned selection and A denotes No Selection.

Task	Median C	Median A	A/C	Mean C	Mean A
VideoUnmask	2.23	40.47	18.1×	3.87	41.41
VideoRepick	4.38	52.66	12.0×	5.51	56.61

G.3 Language-conditioned grounding.

The No Language intervention replaces entity phrases with role-level queries: covers, containers, locations, and regions use *a container*; controls, buttons, and waypoints use *a control*; pegs use *a peg*; remaining entities use *an object*. For example, *a green cube* becomes *an object*, while a PatternLock request containing *a stick*, *a button*, *a pad*, *a marker*, and *a circle* becomes *an object* followed by four *a control* queries. Concept selection, required counts, role assignments, detector, keyframe budget, and policy instruction remain fixed. Policy evaluation includes episodes with empty or partial memory resulting from unavailable groundings.

Offline evaluation measures agreement with pipeline-generated reference records extracted from stored full-memory outputs before the query intervention. Target-location agreement compares exact 8×8 cells: a nonzero Reader coordinate c is mapped to $1 + \lfloor (c - 1)/4 \rfloor$. Sequence agreement requires the same length and exact cell agreement at every sequence position. VideoUnmask uses the first 100 archived episodes, with 98/100 matches for specific queries and 20/100 for generic queries. PatternLock uses 98: episodes 29 and 83 lack the required reference row at their retrieval frame and are excluded from both conditions. Matches are 86/98 and 0/98. Separately, 50 matched closed-loop episodes give 43/50 versus 12/50 for VideoUnmask and 36/50 versus 1/50 for PatternLock.

Table 7 reports both the reference-matching counts and the separate closed-loop evaluation counts.

Table 7: Language-grounding results. C uses specific entity queries; A uses generic role-level queries. Matches denote agreement with pipeline-generated reference records for the target cell (VideoUnmask) or complete ordered sequence (PatternLock). Successes denote closed-loop task completion.

Task	Matches C	Matches A	Successes C	Successes A
VideoUnmask	98/100	20/100	43/50	12/50
PatternLock	86/98	0/98	36/50	1/50

H VLM-based Concept Extraction

H.1 Dataset and supervision.

We construct a dataset of 504 recorded RoboMME videos from six tasks: VideoRepick, PickHighlight, PickXtimes, StopCube, VideoPlaceOrder, and VideoPlaceButton. The dataset covers eight concepts: picked-object identity, highlighted-object sets, current holding and highlighted states, pick-place counts, region-entry counts, placement order, and placement relations around button presses. We use 365 training, 44 validation, and 95 test videos, keeping scene-seed groups within a single split. Reference labels are generated from independent simulator replays using observed physical events, rendered highlights, and button states. Replay alignment is verified against recorded RGB, depth, robot poses, and camera calibration. Labels are algorithmically generated and audited, rather than exhaustively hand-annotated.

Historical concepts use complete video clips, whereas current-state concepts query a named object at a video-prefix endpoint. For state supervision, we select one stable positive and one stable negative endpoint per eligible video, producing 808 queries from 404 of the same videos: 590 training, 68 validation, and 150 test queries. Holding labels require bilateral finger contact and object lift; highlighted-state labels require an active visible ring. Both states must persist over five frames. All derived queries retain their source video’s split. Each input contains a concept definition and 64 uniformly sampled front-camera RGB frames at 256×256 resolution, restricted to the queried prefix for current-state questions. Objects are referenced through first-frame appearance or spatial descriptions; reference boxes, instance IDs, and simulator states are not provided to the model.

H.2 Training.

We compare two matched formulations: *Structured* generates JSON records, and *QA* generates concept-specific textual answers, including yes/no for state queries. Both use identical visual inputs, semantic supervision, and sample order within each comparison. We train separate adapters for historical and current-state concepts under each formulation, all initialized from the original Qwen3-VL-8B-Instruct checkpoint. Only language-model attention LoRA parameters are updated (q, k, v, o projections; rank 8, scaling 16, dropout 0); all remaining parameters are frozen. Training uses BF16, answer-token cross-entropy, AdamW with a constant learning rate of 10^{-4} , weight decay 0.01, and gradient clipping at 1.0. Historical-concept training comprises 915 updates and 1,830 video presentations per formulation, including one distributed-sampling padding example per epoch. Current-state training comprises 1,475 updates and 2,950 query presentations per formulation. Each checkpoint is selected by the lowest validation answer-token negative log-likelihood, without test-based selection.

I Real-Robot Experimental Details

I.1 Tasks and data.

We evaluate *ScoopPour* and *CupSwitch* using the right arm of a YAM robot. *ScoopPour* requires scooping red beans from a tray with a yellow scoop and pouring them onto a plate. *CupSwitch* requires moving a purple cup between two plates and back; one round trip counts as one cycle. Demonstrations record top-view and right-wrist RGB images (640×480), right-arm state, and control commands at a nominal 30 Hz. State and action each contain six joint coordinates and one gripper coordinate.

After screening, each task uses 59 demonstrations: 80,203 frames for *ScoopPour* and 81,200 for *CupSwitch*. The respective two-/three-cycle splits are 24/35 and 29/30 demonstrations. Both methods use the same retained data, with English instructions specifying the task and count. All retained demonstrations are used for fine-tuning. Evaluation covers *ScoopPour* 1–3 $\times$ and *CupSwitch* 2–3 $\times$; the one-cycle *ScoopPour* target is absent from the fine-tuning data.

I.2 Concept adaptation and memory generation.

For *ScoopPour*, we introduce `event.scoop` while reusing entity grounding, event ordering, and counting. OWLv2 grounds the scoop, tray, and plate; top-view tracking estimates location, and wrist-view appearance indicates whether the scoop contains beans. A loaded scoop after visiting the tray establishes an acquisition candidate. A subsequent loaded-to-empty transition above the plate, confirmed across consecutive obser-

uations, completes one cycle. CupSwitch reuses entity, identity, pick-and-place, sequence, and counting concepts to compose two directed transfers into a round trip.

Agent-assisted visual review and user feedback guide development on six ScoopPour and two CupSwitch demonstrations. We inspect six additional demonstrations per task; the ScoopPour recognizer is frozen before this check. Review compares visible states, event evidence, cycle counts, and confirmation timing. Logic tests and evidence-removal replays check event order, repeated counting, and required observations.

The recognizers automatically generate causal, frame-aligned memory for all retained demonstrations. The existing 11-field interface encodes entity locations, current states and relations, observed events, and goal progress. Current states update with observations, while completed events retain their confirmation frames. Each policy input includes only memory available by that frame; the instruction supplies the requested count, and recognized events supply observed progress.

I.3 Policy training.

We train four task-specific policies: Concept Memory and a no-memory $\pi_{0.5}$ baseline for each task. Shared backbones initialize from official pretrained $\pi_{0.5}$ weights and undergo full-parameter fine-tuning. Both methods use identical task data, inputs, and normalization; the baseline omits the memory encoder and tokens. Images are letterboxed to 224×224 , and state/actions use first- and 99th-percentile normalization. Dense-frame sampling supplies 20-step action targets, masking unused dimensions and episode-tail padding.

I.4 Deployment and evaluation.

A remote GPU server predicts 20-step action chunks, and the local robot executes the first 16 steps before replanning. Memory updates from streamed observations at configured rates of 5 Hz for ScoopPour and 10 Hz for CupSwitch and resets at episode boundaries. Both methods use the same execution interface and command-increment limits.

I.5 Results.

Table 8 reports success rates across the five conditions. Concept Memory achieves 68/79 successes (86.08%), versus 5/58 (8.62%) for the baseline, a gain of 77.46 percentage points. Their pooled Wilson 95% confidence intervals are [76.76, 92.04]% and [3.74, 18.64]%, respectively (two-sided Fisher exact test: $p = 4.90 \times 10^{-21}$). Table 9 details the manual outcome annotations.

Table 8: Real-robot success rates by task and target count. Targets count scoop-and-pour cycles for ScoopPour and complete round trips for CupSwitch. Gain is measured in percentage points (pp).

Task	Target	Concept Memory		No-memory $\pi_{0.5}$		Gain (pp)
		Success / Trials	Rate (%)	Success / Trials	Rate (%)	
ScoopPour	1×	16 / 20	80.00	4 / 12	33.33	+46.67
	2×	19 / 23	82.61	0 / 12	0.00	+82.61
	3×	11 / 12	91.67	0 / 11	0.00	+91.67
CupSwitch	2×	11 / 12	91.67	0 / 12	0.00	+91.67
	3×	11 / 12	91.67	1 / 11	9.09	+82.58
<i>Aggregates over reported conditions</i>						
ScoopPour	1–3×	46 / 55	83.64	4 / 35	11.43	+72.21
CupSwitch	2–3×	22 / 24	91.67	1 / 23	4.35	+87.32
Pooled success rate		68 / 79	86.08	5 / 58	8.62	+77.46
Mean over five conditions		–	87.52	–	8.48	+79.04

Reported conditions comprise ScoopPour 1–3× and CupSwitch 2–3×. The mean weights these five conditions equally; the pooled rate weights individual trials equally.

Table 9: Manual real-robot outcome annotations. Numeric entries give the number of trials. Baseline outcomes include successes and failures.

Task	Target	Concept Memory failures	No-memory $\pi_{0.5}$ outcomes	
			Recorded cycle counts	Other outcomes
ScoopPour	1×	4 failures	1 (success): 4; 2: 2; 6: 1	Too many cycles: 2; timeout: 1; freeze: 1; miss: 1
	2×	4 failures	0: 2; 1: 6; 3: 1; 4: 1	Timeout: 2
	3×	1 failure	2: 2; 4: 2	Timeout: 7
CupSwitch	2×	1 timeout	0: 6; prolonged 0-cycle execution: 1	Timeout: 4; miss: 1
	3×	1 timeout	0: 1; 1: 2	Too many cycles: 2; timeout: 3; miss: 1; 3 cycles then timeout: 1; success: 1

Notation: An entry of 2: 4 means four trials annotated with two completed cycles. *Too many cycles* retains annotations without an exact count. *Scoring:* Concept Memory trials marked `fail` or `timeout` are failures; other annotations indicate success. For the baseline, a numeric suffix matching the requested count or an absent suffix indicates success. Textual failure annotations, including timeouts after reaching the target count, indicate failure.